Sri Lanka Institute of Information Technology

IT3021 – DWBI

Assignment 01

Name - J.H.F. Zahra

Student ID - IT23708280

Campus – Malabe

Batch – Y3.S1.WE.DS.01.01

Date of Submission – 30.03.2026

Table of Contents

1. Dataset Selection

1.1 Overview

This project develops a complete data warehousing and business intelligence solution for XYZ Superstore, a retail organization operating across the United States and Canada. The dataset encompasses four years (2021–2024) of sales transactions totaling 10,194 records across three product categories (Office Supplies, Furniture, Technology) and three customer segments (Consumer, Corporate, Home Office). The dataset was sourced from Kaggle (see References).

1.2 Dataset Justification

The Superstore dataset was selected as it represents a realistic OLTP retail scenario with sufficient complexity to demonstrate all required data warehousing concepts. It provides over one year of transactional data, multiple logical entities with clear relationships, natural hierarchies (Category → SubCategory → Product, Year → Quarter → Month → Date), and enough volume to support SSAS cube construction, aggregate analysis, and report generation. The dataset is not an OLAP dataset and does not pre-define fact and dimension structures, making it suitable for dimensional modelling from scratch.

1.3 Source Data Structure

The original dataset exists as a single denormalized CSV file containing 10,194 rows where each row represents one order line item - a single product within an order. The file contains five logical groupings of attributes:

- **Primary Key:** RowID uniquely identifies each order line item
- **Order Information:** OrderID, OrderDate, ShipDate, ShipMode
- **Customer Information:** CustomerID, CustomerName, Segment
- **Location Information:** City, State/Province, PostalCode, Region, Country
- **Product Information:** ProductID, ProductName, Category, SubCategory
- **Transaction Metrics:** Sales, Quantity, Discount, Profit

This denormalized structure contains significant redundancy - customer, product, and location data repeat across multiple rows. For example, when a customer places an order containing three products, their name, segment, and address appear in three separate rows. This redundancy is characteristic of flat file extracts and necessitates the dimensional modelling approach implemented in the data warehouse.

1.4 Conceptual Data Model

Figure 1.1 illustrates the conceptual entity model underlying the physical structure. Four logical entities - Customer, Order, Order_Line, and Product - and their relationships are identified.

- A Customer places one or many Orders,
- An Order contains one or many Order Lines
- Each Order Line references one Product.

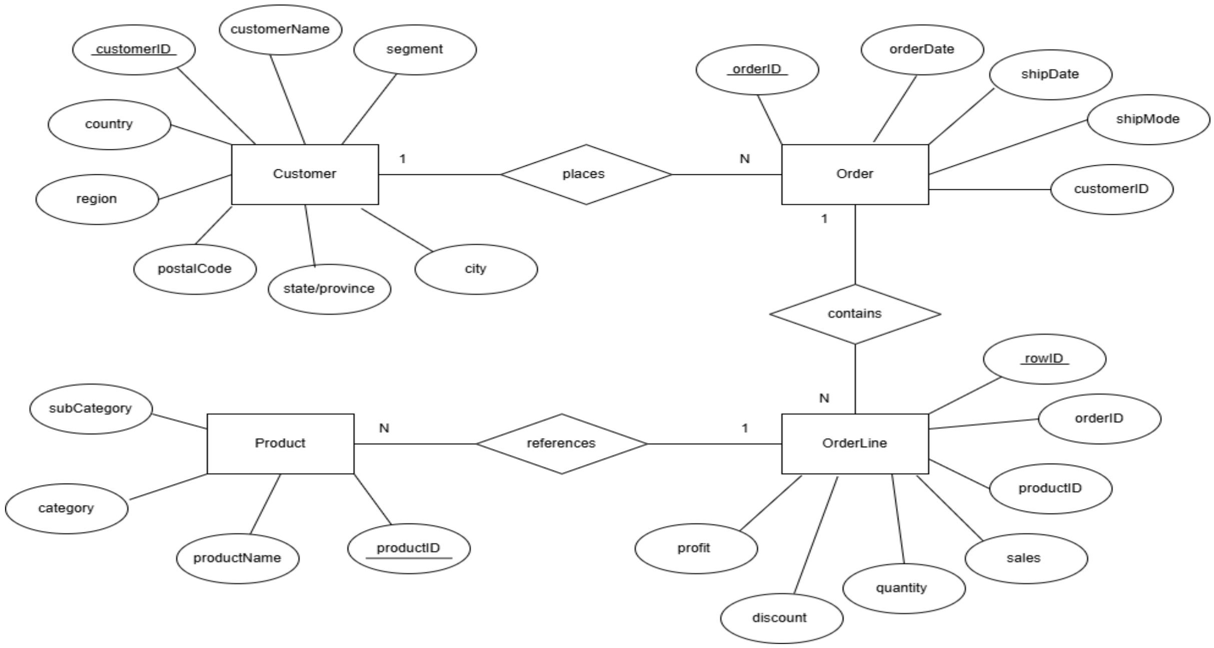


Figure 1.1- ER Diagram illustrating the conceptual data model of the Superstore source dataset showing four entities and their relationships

1.5 Project Objectives

The primary objective of this assignment is to transform this operational (OLTP) dataset into a structured data warehouse using dimensional modeling principles (star schema) to support advanced analytics and business intelligence capabilities. Key deliverables include:

1. Data Warehouse Design: Star schema with 4 dimensions and 1 fact table with a grain of one row per order line item
2. ETL Development: SSIS packages extracting from multiple sources with SCD Type 2 implementation
3. Advanced Features: Slowly changing dimensions for customer tracking and accumulating fact tables for transaction lifecycle monitoring

The solution enables comprehensive analytics including sales performance, customer behavior, product trends, geographic analysis, and operational efficiency metrics, providing actionable insights for strategic decision-making.

2. Data Source Preparation

2.1 Overview

The original single-file dataset was strategically split into three distinct source types to simulate a realistic enterprise data environment and satisfy the requirement of at least two heterogeneous data sources. This approach mirrors common enterprise scenarios where transactional systems, CRM exports, and business-managed reference files coexist.

2.2 Source Types

2.2.1 SQL Server Database - Superstore_Source_DB

The core transactional data is loaded into a SQL Server database table. This represents the high-velocity operational system maintained by IT.

Table	Attributes
StgOrders	RowID, OrderID, OrderDate, ShipDate, ShipMode, CustomerID, ProductID, Sales, Quantity, Discount, Profit

2.2.2 CSV Flat File - Superstore_Customer_Source.csv

Customer master data is maintained as a CSV flat file, simulating a periodic export from an external CRM system.

File	Attributes
Superstore_Customer_Source.csv	CustomerID, CustomerName, Segment, City, State/Province, PostalCode, Region, Country

2.2.3 Excel File - Superstore_Product_Source.xlsx

Product reference data including the category hierarchy is maintained in an Excel workbook, simulating business-managed reference data updated by product managers.

File	Attributes
Superstore_Product_Source.xlsx	ProductID, ProductName, Category, SubCategory

2.2.4 CSV Flat File - txn_completions.csv

A separate CSV file is used for the accumulating fact table update process. It contains order completion timestamps used to update FactSales after initial loading.

File	Attributes
txn_completions.csv	OrderID, accm_txn_complete_time

2.3 Source-to-Warehouse Mapping

The table below summarises how each source feeds into the data warehouse:

Source	Format	Target DW Table
Superstore_Source_DB	SQL Server	FactSales, DimShipMode
Superstore_Customer_Source.csv	CSV	DimCustomer
Superstore_Product_Source.xlsx	Excel	DimProduct
txn_completions.csv	CSV	FactSales (accumulating update)
DateMaster.sql	SQL Script	DimDate

2.4 Splitting Rationale

The data was distributed across source types based on the following considerations:

- **Transactional vs. reference nature:** High-frequency transactional order data is best suited to a relational database, while customer and product reference data can be maintained as files.

- **Update frequency:** Order data is updated continuously (SQL), customer data is refreshed periodically (CSV), and product hierarchy data changes infrequently (Excel).
- **ETL complexity:** The multi-source approach enables demonstration of OLE DB connections, flat file parsing, Excel workbook navigation, cross-source lookups, and data enrichment transformations within SSIS.

3. Solution Architecture

Source Systems: Three source types feed the pipeline - an SQL Server table for orders, a CSV file for customer data, and an Excel file for product data.
Staging Database: Load_Staging.dtsx extracts from all three sources and lands the raw data into a staging database, decoupling source systems from the warehouse.
SSIS ETL - Load_DataWarehouse.dtsx: Reads from staging, applies transformations (SCD for DimCustomer, lookups, derived columns) and loads the star schema dimensions and fact table.
SSIS ETL - Update_AccumulatingFact.dtsx: A separate package that reads order completion timestamps from a CSV and updates FactSales with accm_txn_complete_time and the calculated txn_process_time_hours.
Superstore_DW - Star Schema: Central data warehouse with FactSales surrounded by DimCustomer, DimProduct, DimShipMode and DimDate, implementing a star schema with SCD on DimCustomer.
SSAS Cube: Multidimensional cube enabling OLAP analysis with hierarchies across product category and date dimensions.
SSRS / Power BI: Reporting layer for dashboards and business insights such as sales trends and profit analysis by region and segment.

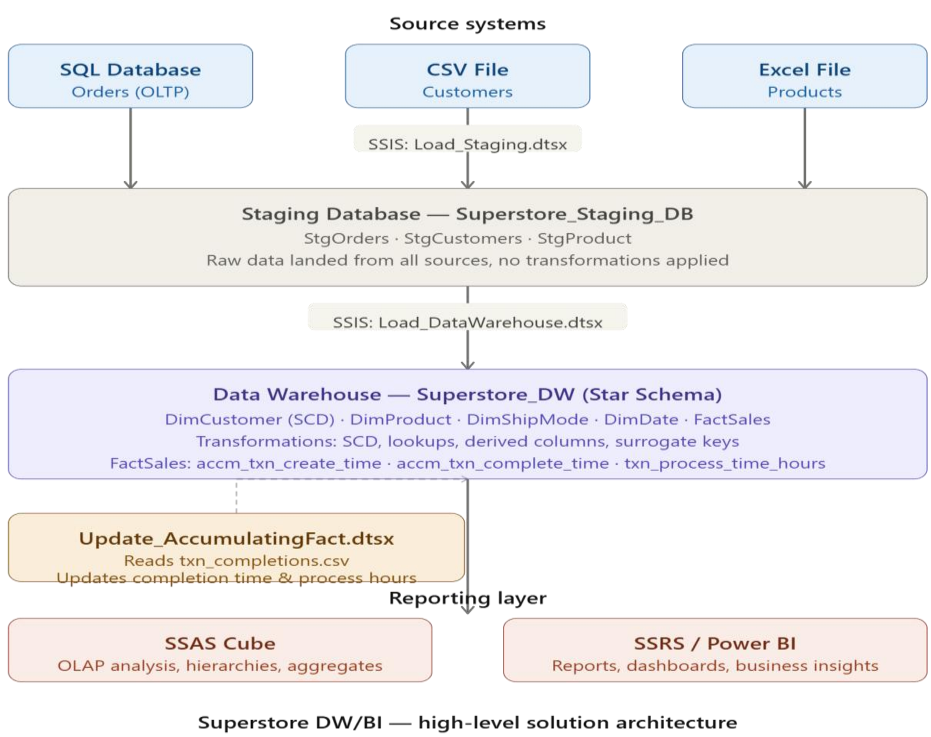


Figure 3.1- High-level DW/BI solution architecture diagram illustrating the end-to-end data flow from heterogeneous source systems through the staging database, ETL processing, data warehouse and reporting layer

4. Data warehouse design & development

4.1 Schema Design

The data warehouse follows a star schema design with one central fact table (FactSales) surrounded by four dimension tables - DimCustomer, DimProduct, DimShipMode and DimDate. The grain of the fact table is one row per order line item, meaning each row represents a single product sold within a specific order.

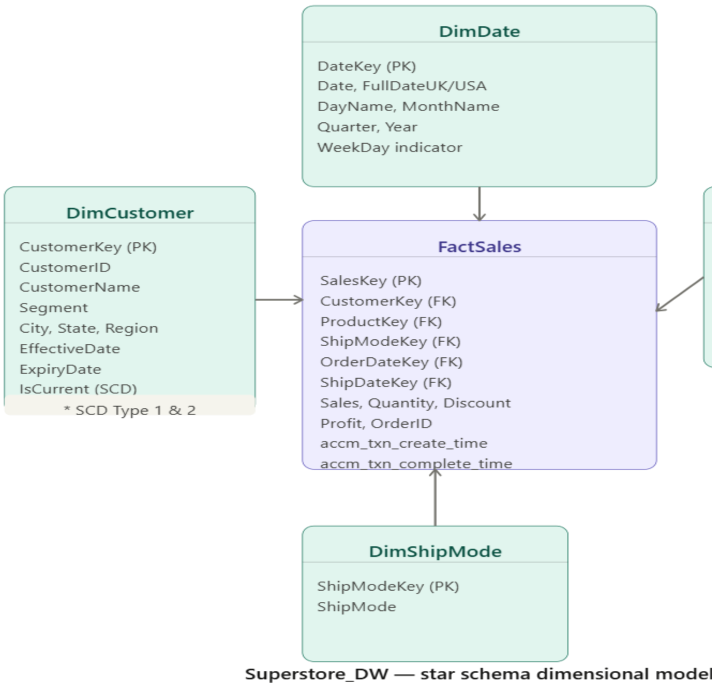


Figure 4.1 - Star schema dimensional model of Superstore_DW showing FactSales and four dimension tables

4.2 Dimension Tables

DimCustomer stores customer demographic and geographic information. It implements a Slowly Changing Dimension (SCD Type 2) to track historical changes to customer attributes over time using EffectiveDate, ExpiryDate and IsCurrent columns.

DimProduct stores product catalog information including the category hierarchy (Category → SubCategory → ProductName), which supports drill-down analysis in SSAS.

DimShipMode is a simple lookup dimension storing the four shipping modes available (First Class, Second Class, Standard Class, Same Day).

DimDate is a calendar dimension populated from 1990 to 2099 supporting date hierarchies (Year → Quarter → Month → Day) for time-based analysis.

4.3 Fact Table - FactSales

FactSales is the central fact table containing surrogate foreign keys to all four dimensions along with the following measures: Sales, Quantity, Discount, Profit, and ProfitMargin. It also contains three accumulating columns (accm_txn_create_time, accm_txn_complete_time, txn_process_time_hours) explained in Task 6.

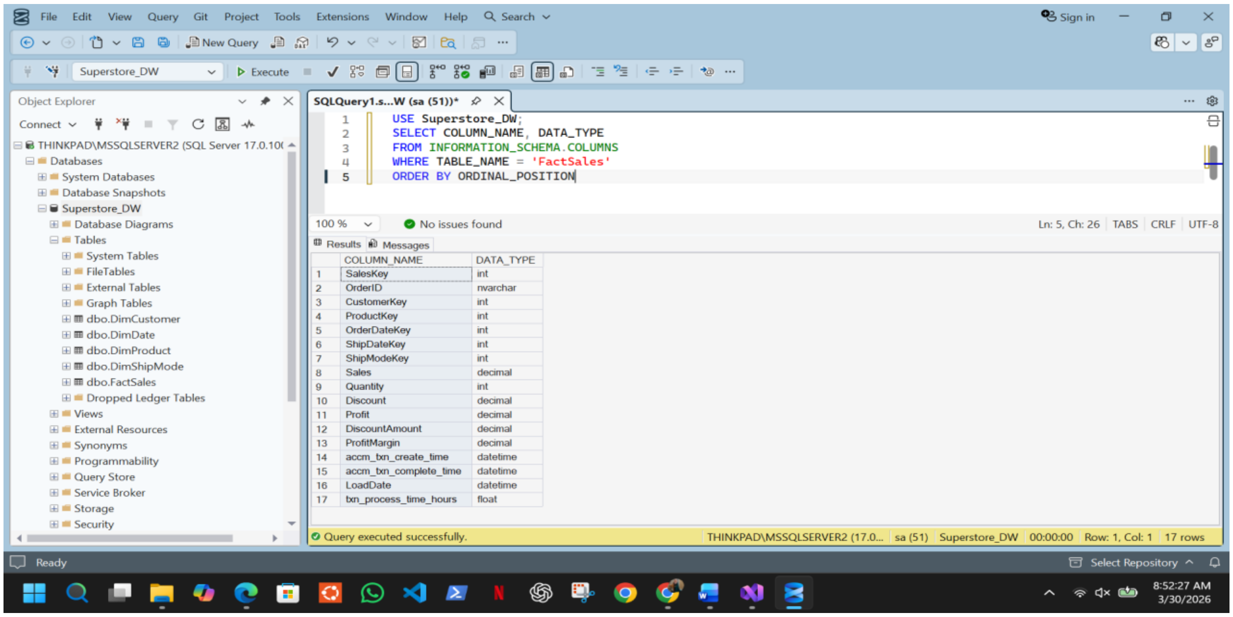


Figure 4.3 - FactSales table structure showing surrogate keys, measures and accumulating columns

4.4 Slowly Changing Dimension - DimCustomer

DimCustomer implements SCD Type 2, meaning when a customer's attributes change (e.g. segment or address), a new row is inserted rather than updating the existing one. The original row's ExpiryDate is set to the change date and IsCurrent is set to 0, while the new row gets IsCurrent = 1 and a NULL ExpiryDate. This preserves the full history of customer changes over time.

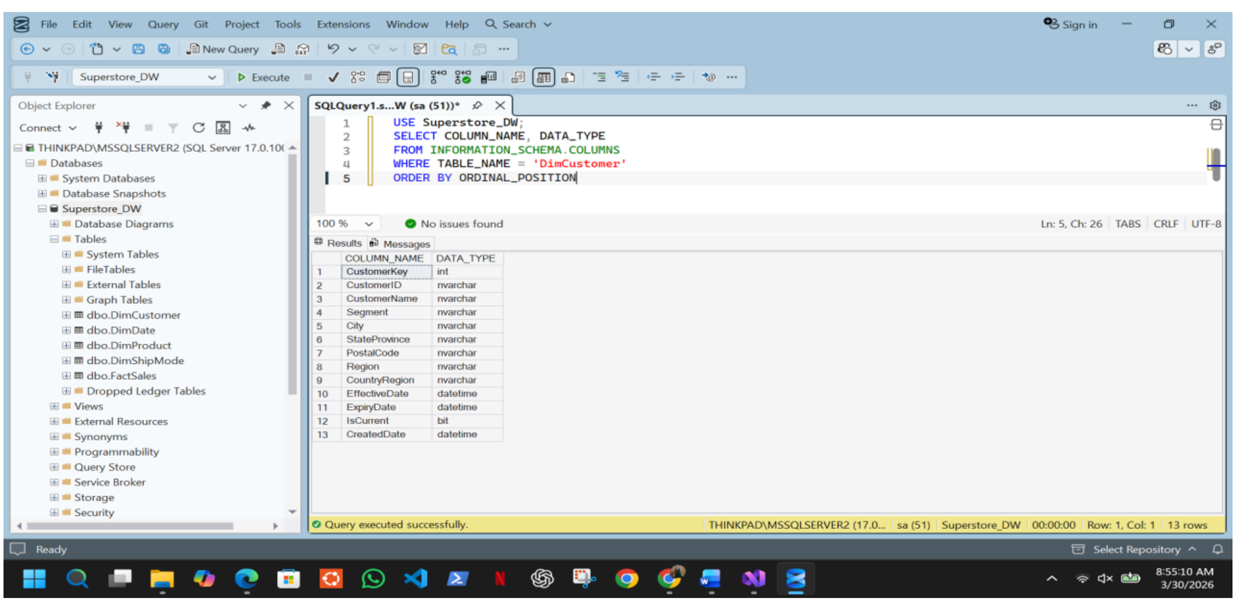


Figure 4.4 - DimCustomer table structure implementing Slowly Changing Dimension Type 2

4.5 Design Assumptions

- Ship Mode is treated as a separate dimension rather than an attribute of FactSales since it is a repeating categorical value suitable for grouping and filtering.
- The date dimension covers 1990–2099 to accommodate historical and future date ranges.
- ProductID from the source is used as the natural key in DimProduct with a system-generated surrogate key (ProductKey) as the primary key.
- NULL values for StockLevel, ReorderPoint and SupplierName in DimProduct are acceptable as this data was not available in the source. The columns are retained in the schema for future data integration.

5. ETL Development

5.1 Overview

The ETL process is split across two SSIS packages executed in sequence. Load_Staging.dtsx extracts raw data from all three source types and lands it into the staging database without any transformation. Load_DataWarehouse.dtsx then reads from staging, applies all transformations, and loads the star schema.

5.2 Load_Staging.dtsx:

This package contains three sequential Data Flow tasks in the Control Flow. Each task is preceded by a truncate operation in the Event Handler to ensure idempotent execution - meaning the package can be re-run safely without duplicating data.

- **Extract Orders to Staging** reads from the SQL Server source database using an OLE DB Source and loads directly into StgOrders.
- **Extract Customers to Staging** reads from the CSV flat file using a Flat File Source and loads into StgCustomers.
- **Extract Product to Staging** reads from the Excel workbook using an Excel Source and loads into StgProduct.

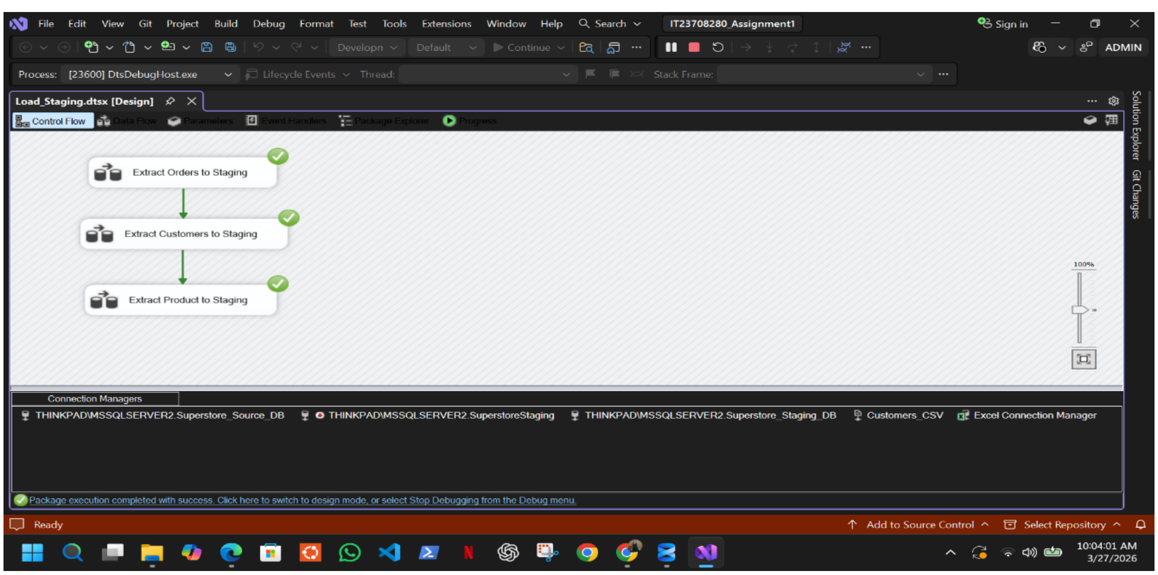


Figure 5.1 - Load_Staging.dtsx Control Flow showing sequential execution of three data extraction tasks

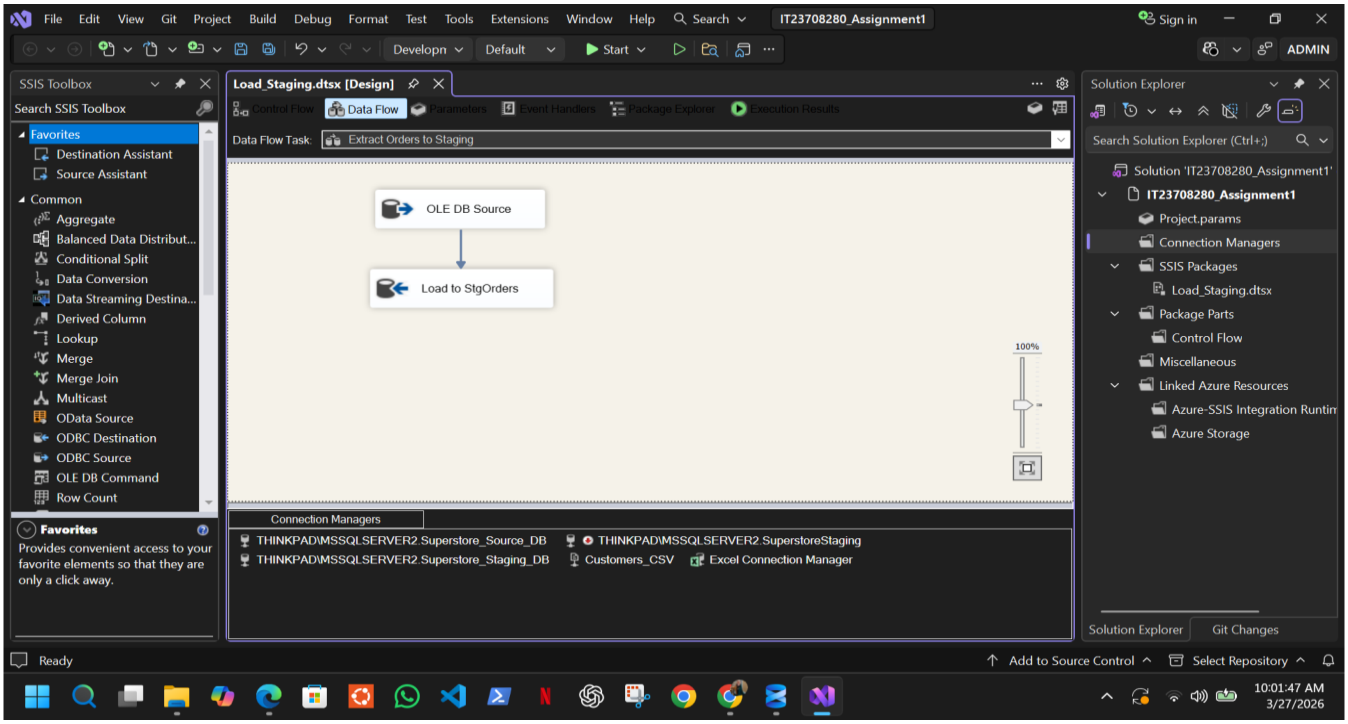


Figure 5.2 - Data Flow for Extract Orders to Staging task reading from Superstore_Source_DB

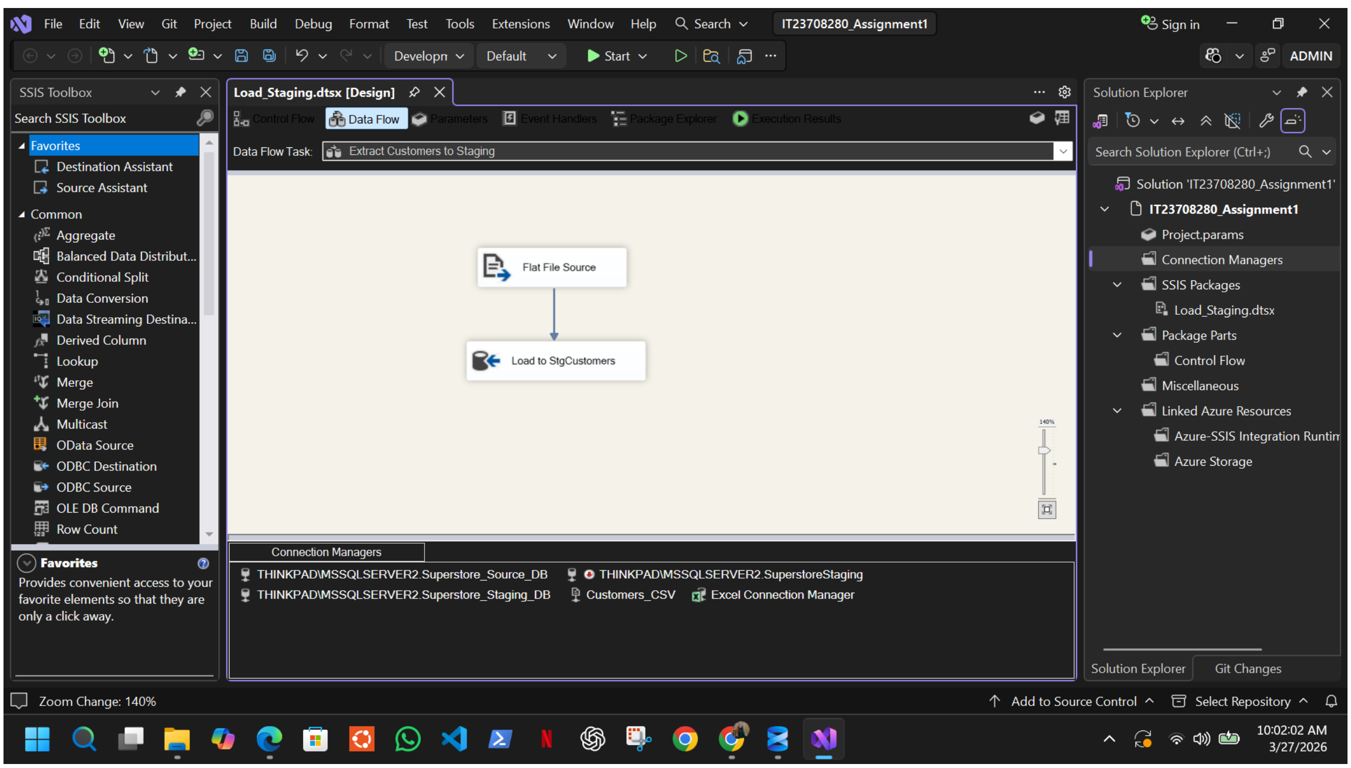


Figure 5.3 - Data Flow for Extract Customers to Staging task reading from CSV flat file

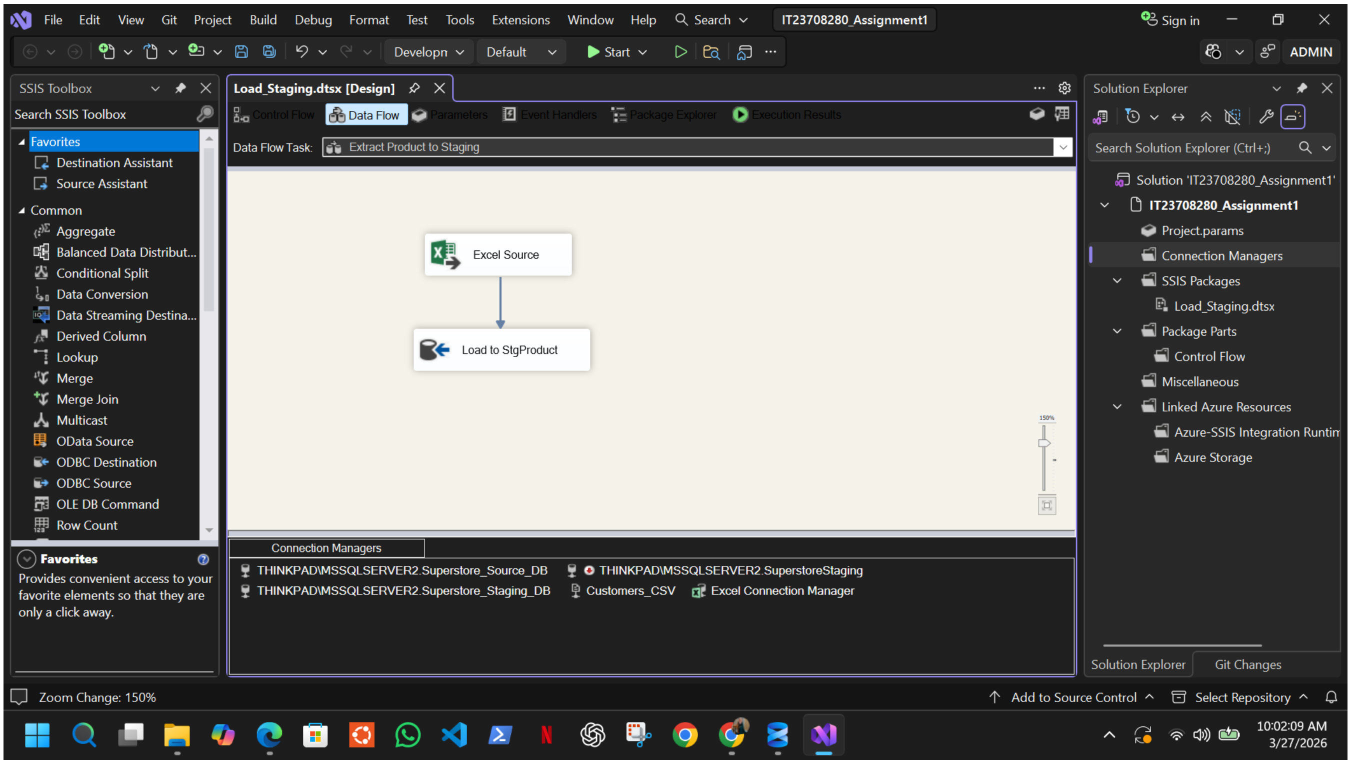


Figure 5.4 - Data Flow for Extract Product to Staging task reading from Excel workbook

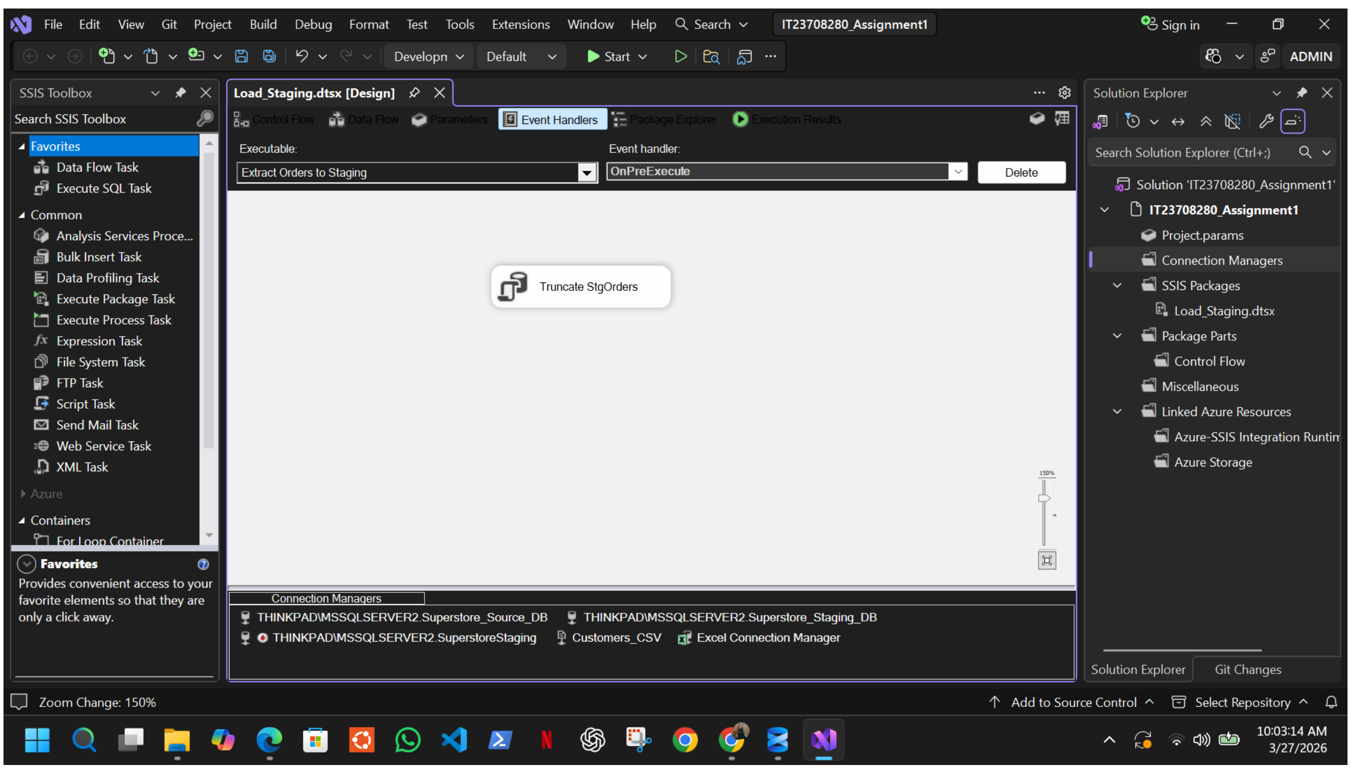


Figure 5.5 - Event Handler configured to truncate staging table before each load to ensure idempotent execution

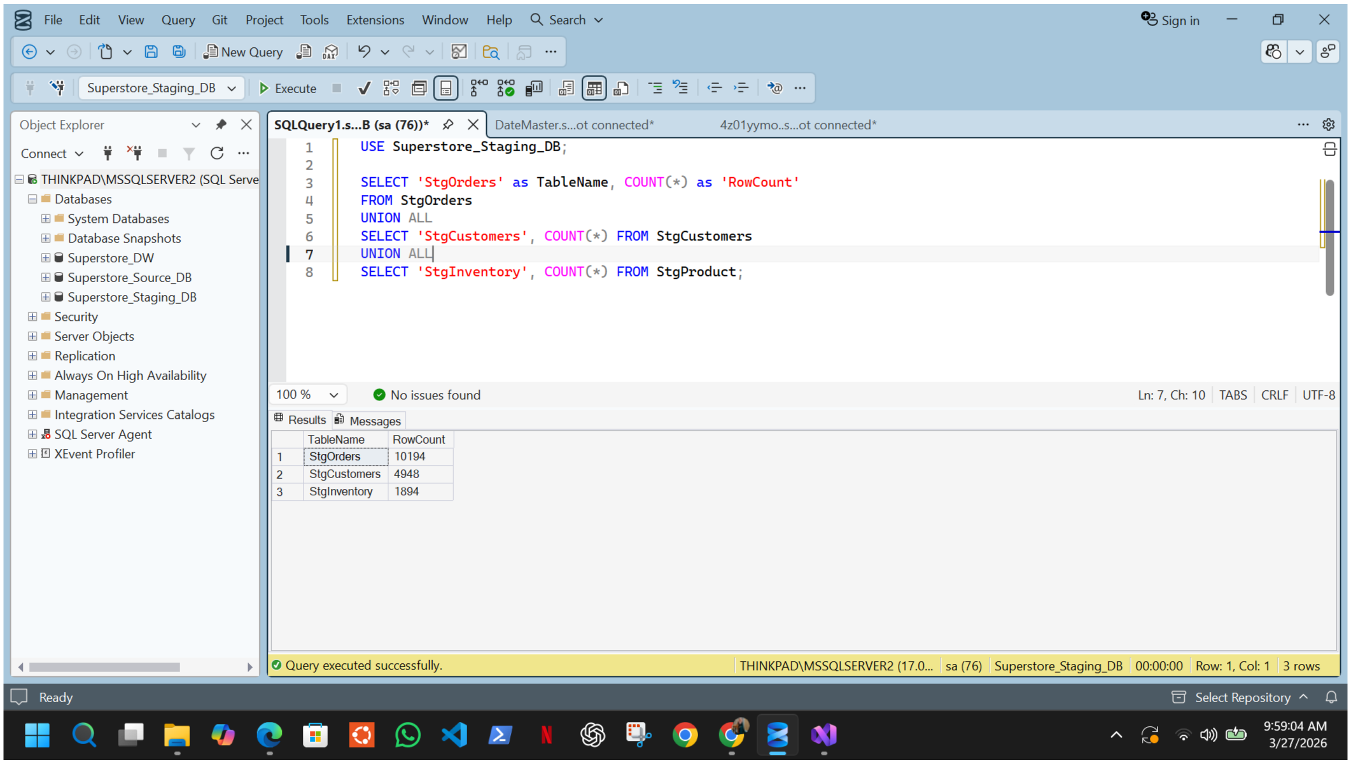


Figure 5.6 - Row count validation confirming successful load of all records into staging tables

5.3 Load_DataWarehouse.dtsx:

This package contains four sequential Data Flow tasks loaded in dependency order - dimensions are loaded before the fact table to ensure surrogate keys are available for lookup.

- **Load DimShipMode** reads distinct ship modes from staging, applies a Sort transformation to remove duplicates, and loads into DimShipMode.
- **Load DimProduct** reads product data from staging, applies a Sort transformation, and loads into DimProduct.
- **Load DimCustomer with SCD** implements SCD Type 2 using the Slowly Changing Dimension component. The pipeline includes Sort, Data Conversion, SCD component, Derived Column, OLE DB Command, Union All and Insert Destination components to handle both new and changed customer records.
- **Load FactSales** reads order line data from staging and performs multiple Lookup transformations to resolve surrogate keys for CustomerKey, ProductKey, ShipModeKey, OrderDateKey and ShipDateKey. A Data Conversion transformation handles data type mismatches and a Derived Column sets the accm_txn_create_time to the current system timestamp. The resolved row is then written to FactSales via OLE DB Destination.

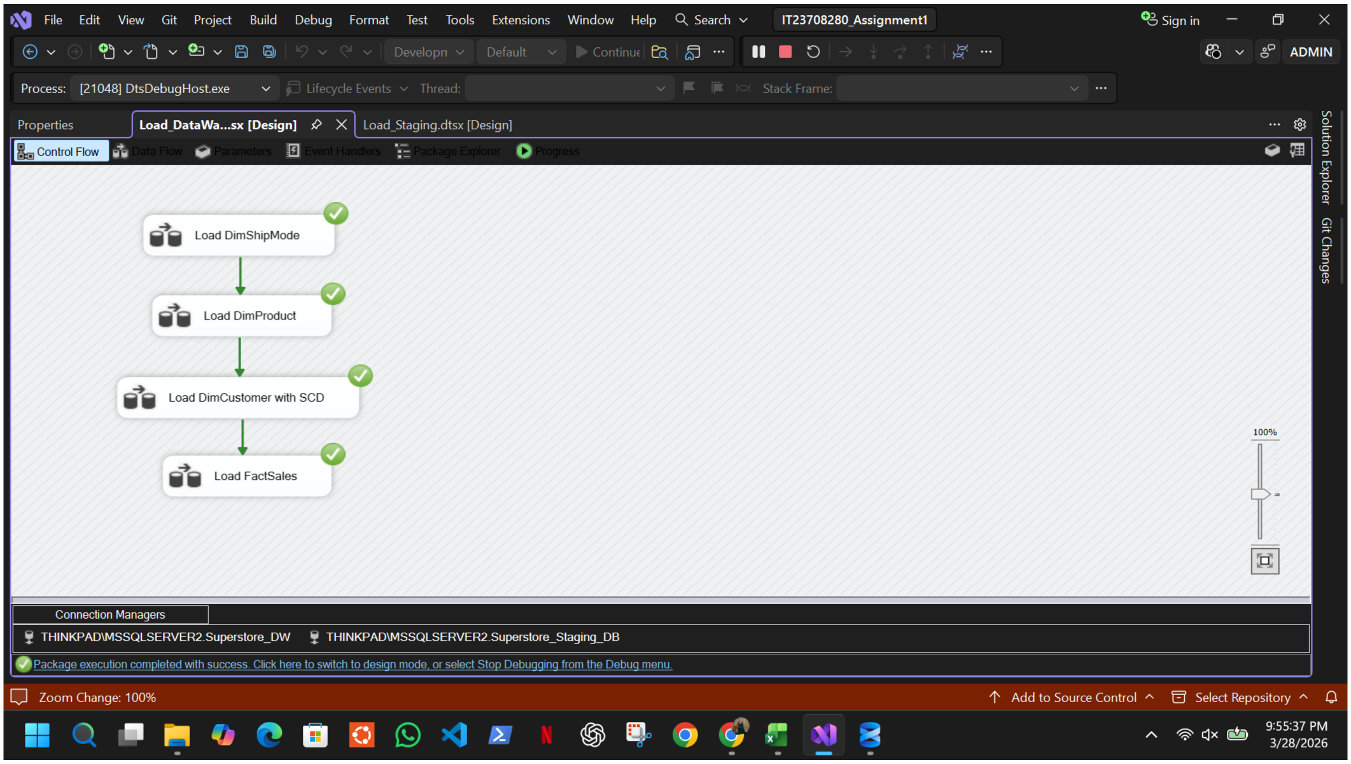


Figure 5.7 - Load_DataWarehouse.dtsx Control Flow showing sequential dimension and fact loading order

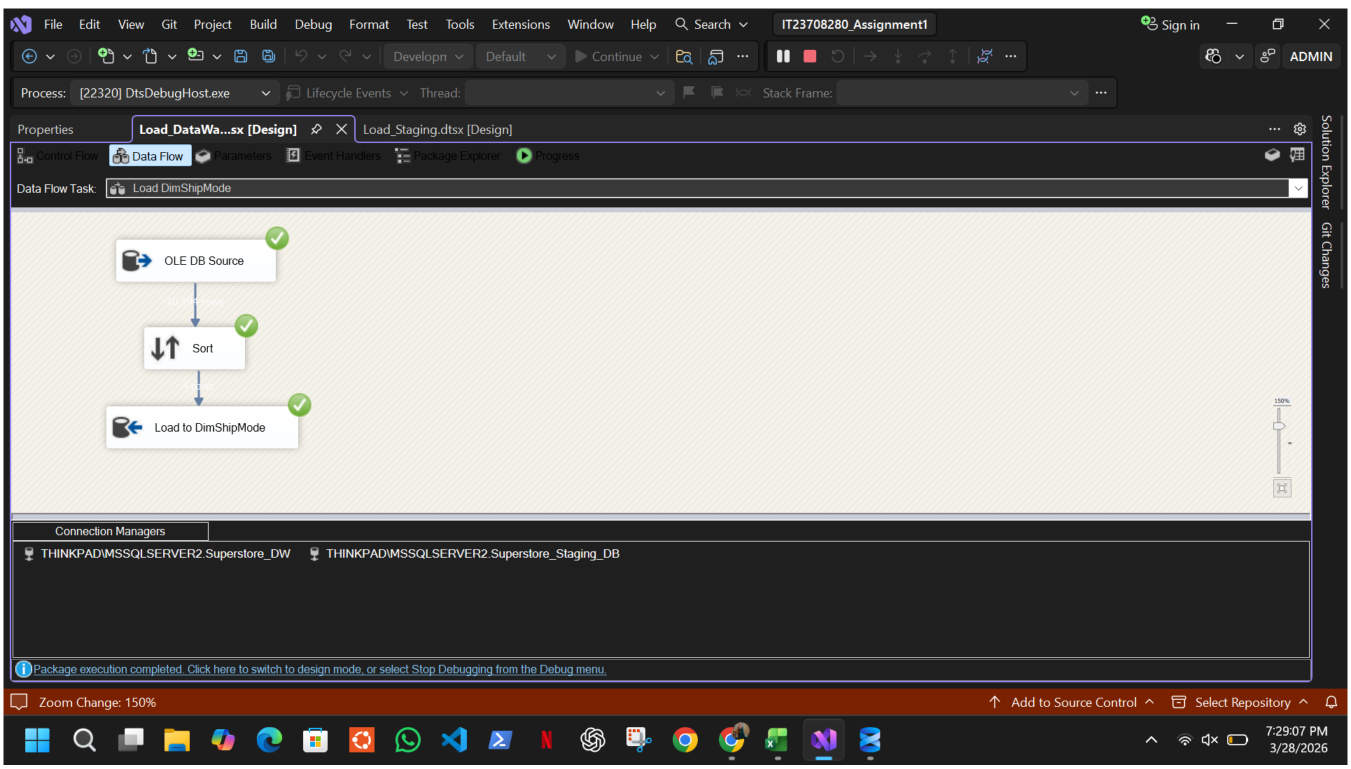


Figure 5.8 - Data Flow for Load DimShipMode task showing sort transformation before destination load

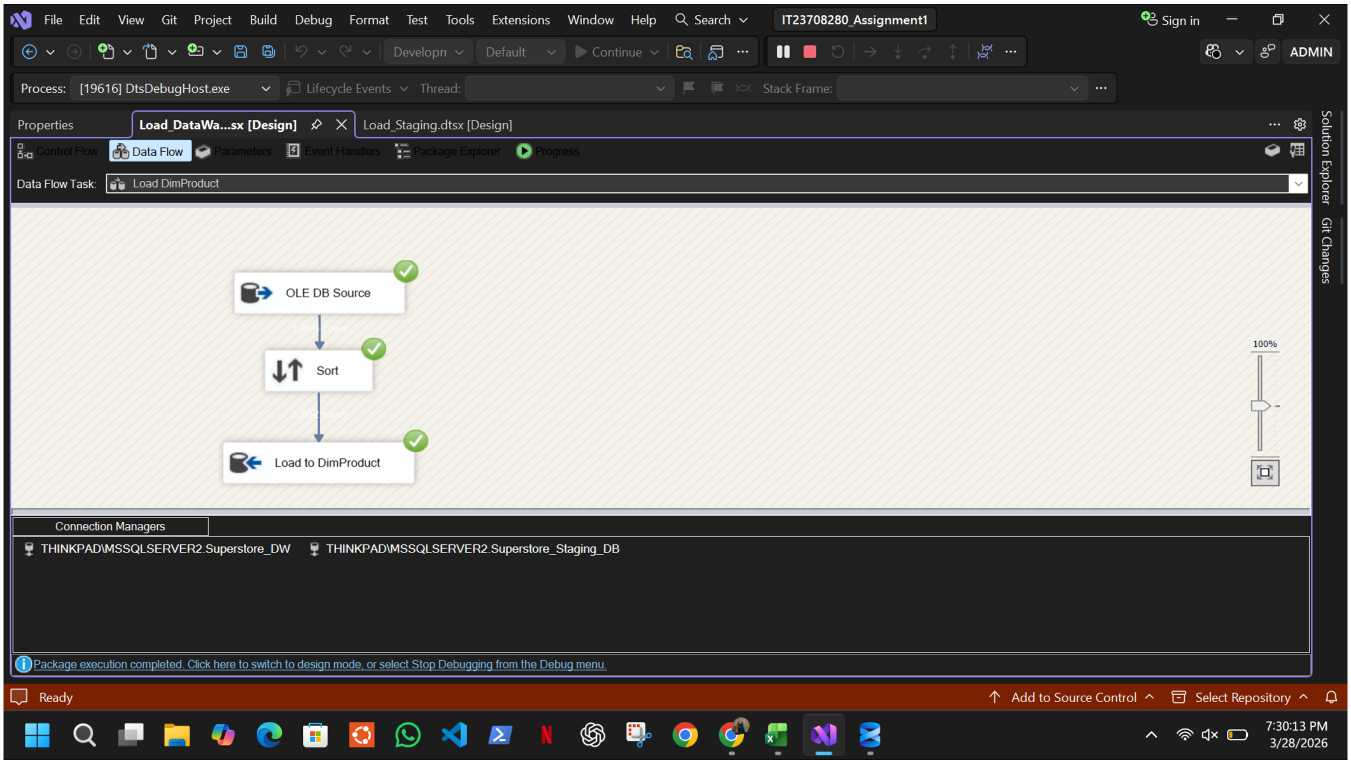


Figure 5.9 - Data Flow for Load DimProduct task showing sort transformation before destination load

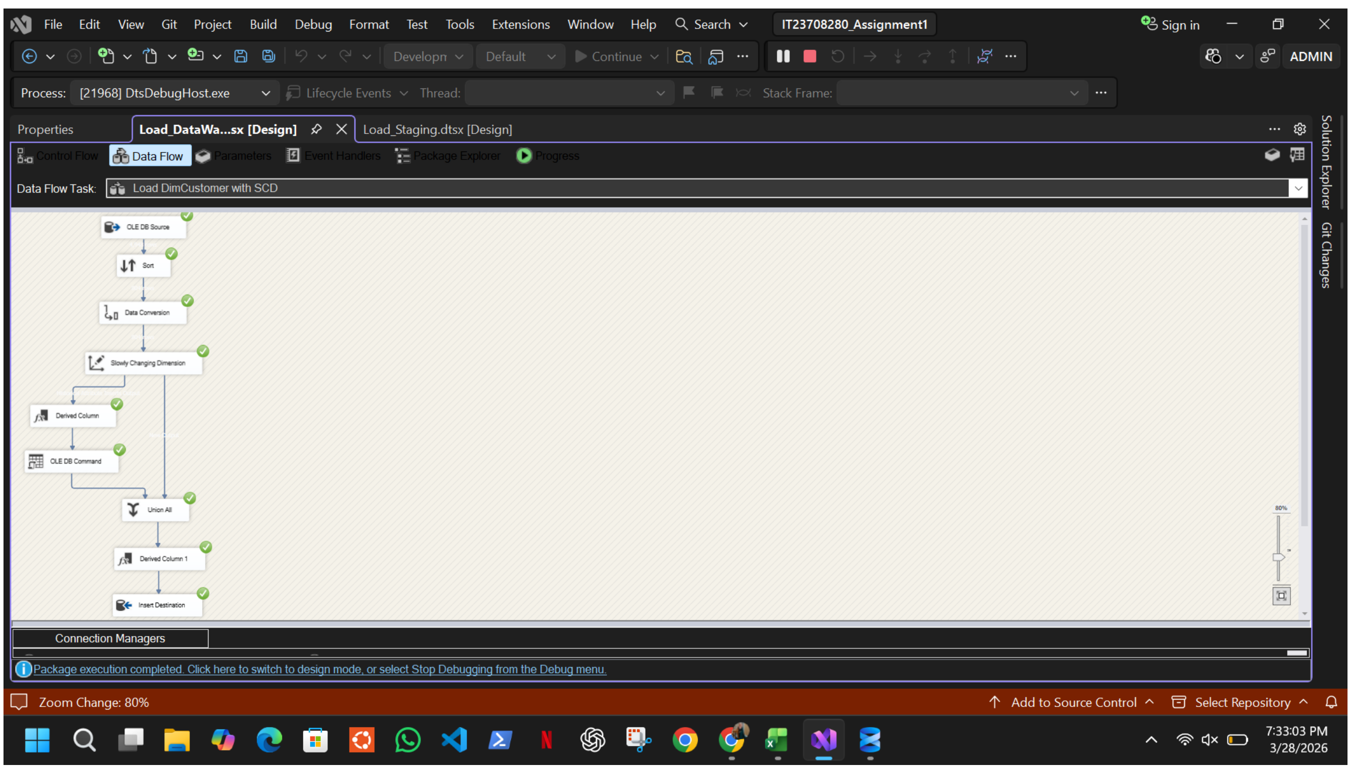


Figure 5.10 - Data Flow for Load DimCustomer implementing SCD Type 2 with Slowly Changing Dimension component, Derived Column, OLE DB Command and Union All

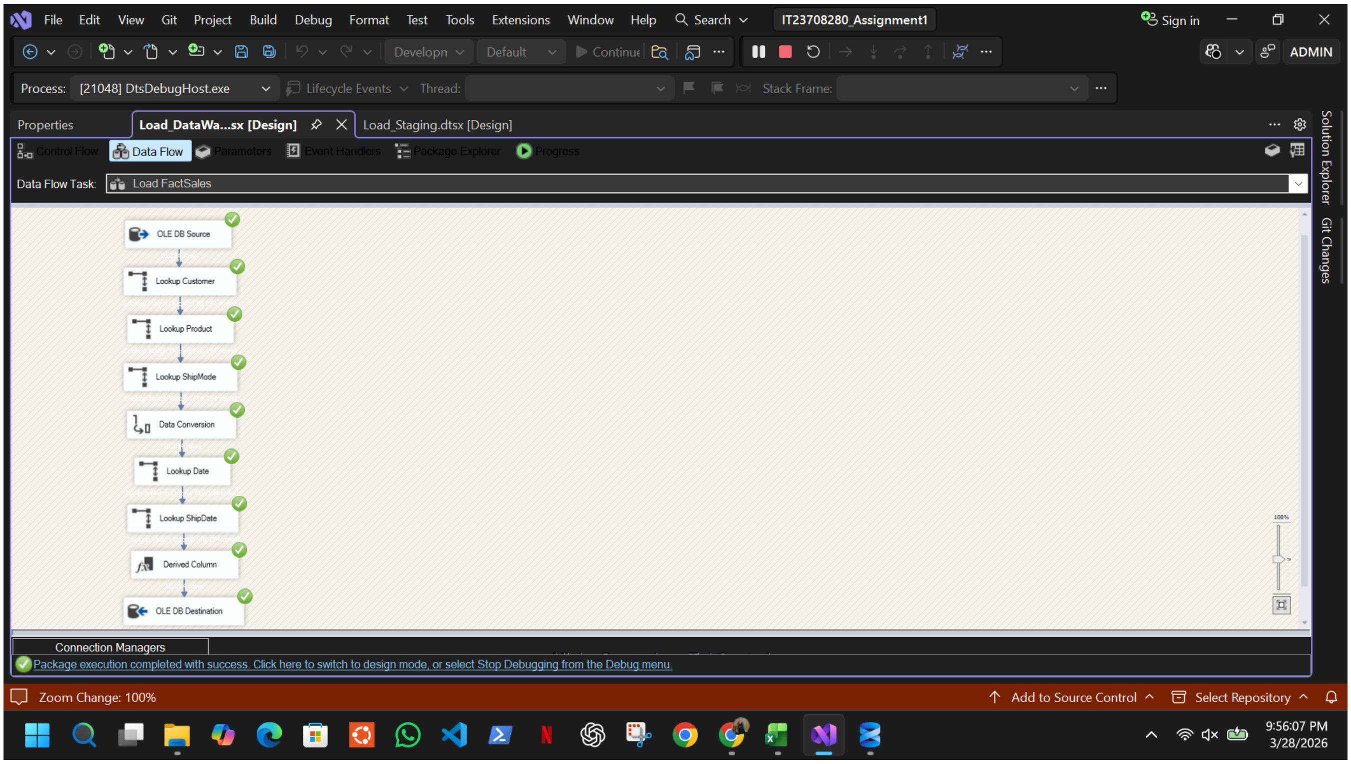


Figure 5.11 - Data Flow for Load FactSales showing lookup transformations for all dimension keys, data conversion and OLE DB destination

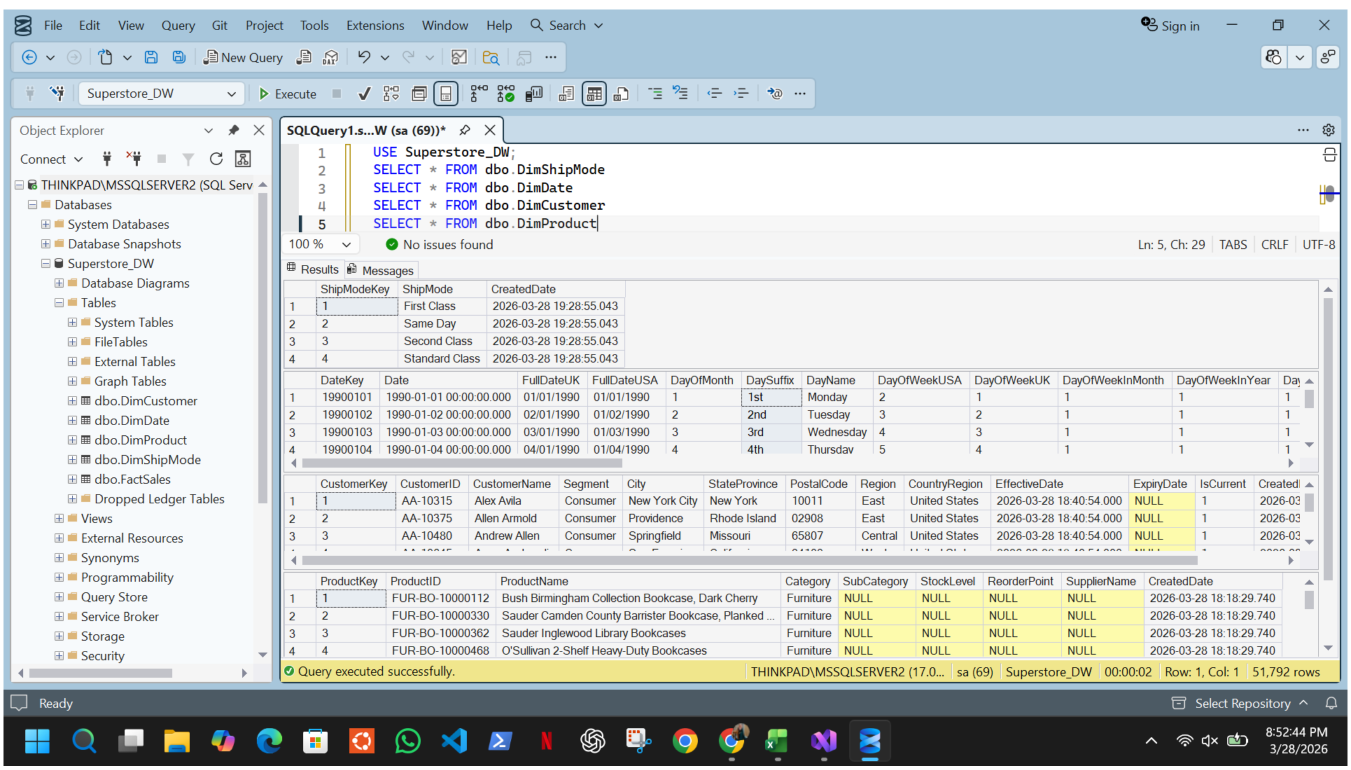


Figure 5.12 - Validation query results confirming successful population of all four dimension tables in Superstore_DW - DimShipMode, DimDate, DimCustomer and DimProduct

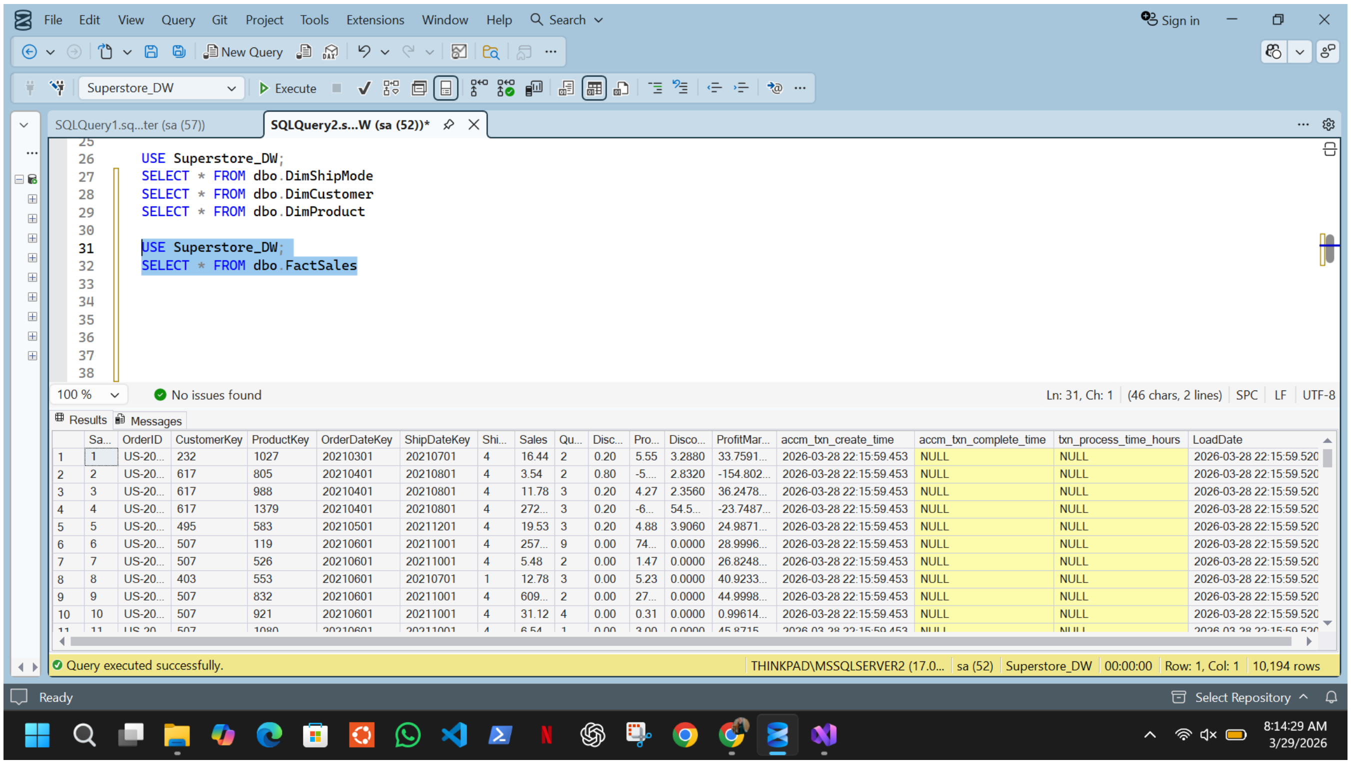


Figure 5.13 - Validation query confirming FactSales loaded with 10,194 rows including accm_txn_create_time populated

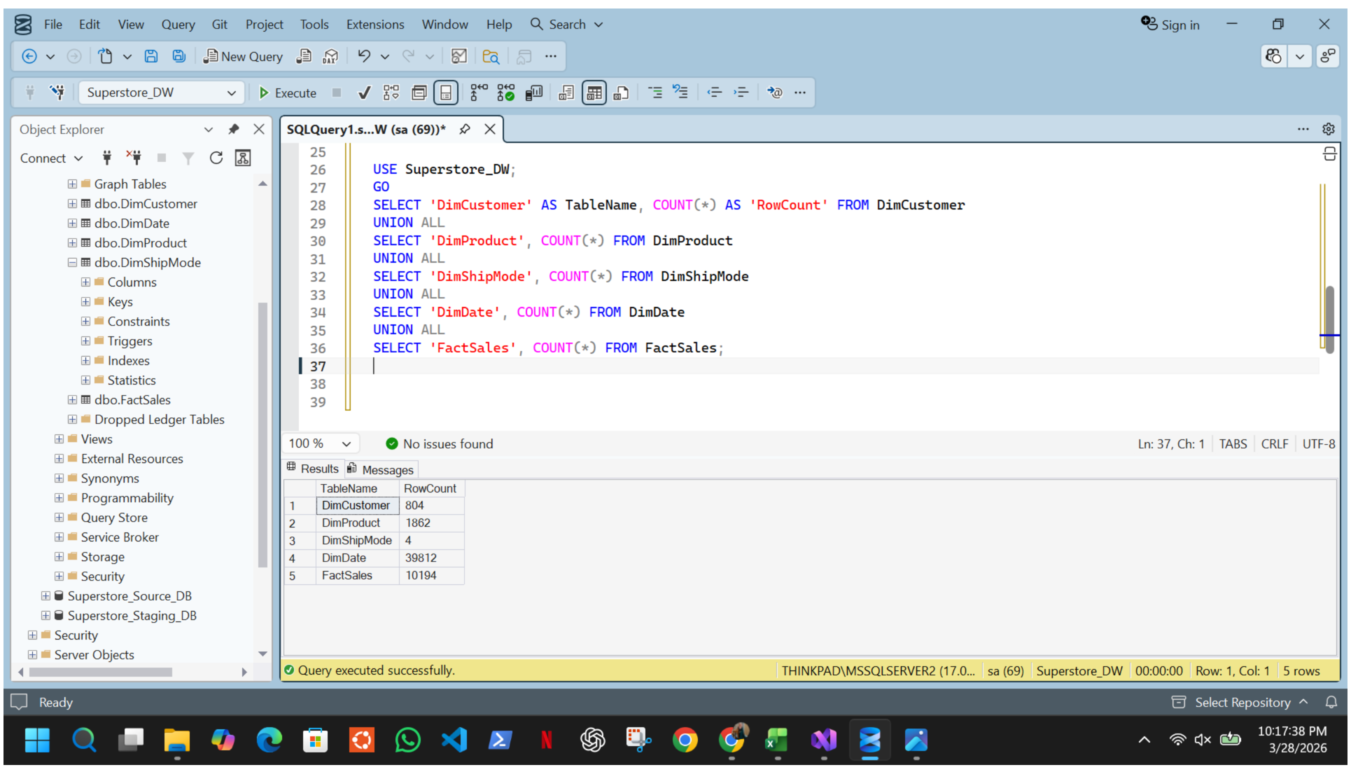


Figure 5.14 - Row count validation confirming successful population of all dimension and fact tables in Superstore_DW

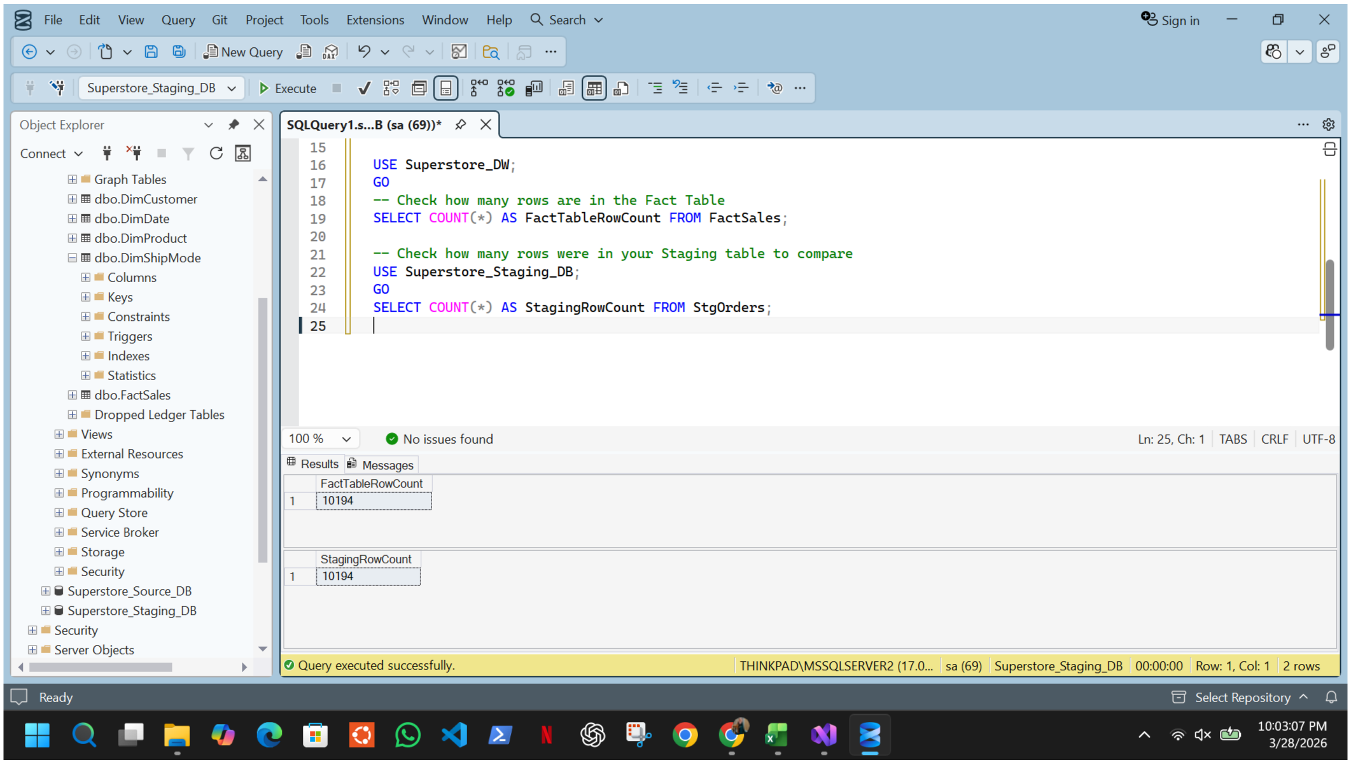


Figure 5.15 - Row count validation confirming FactSales (10,194 rows) matches StgOrders (10,194 rows), verifying no records were lost during the ETL transformation and loading process

6. Accumulating Fact Table

6.1 Concept

An accumulating fact table represents a business process that has a defined lifecycle with multiple stages. Unlike a regular snapshot fact table which is write-once, an accumulating fact table is updated over time as the process progresses through its stages. In this project, a sales order is created at one point in time but may take several days to be fully processed and completed.

6.2 Additional Columns

Three columns were added to FactSales to support this pattern:

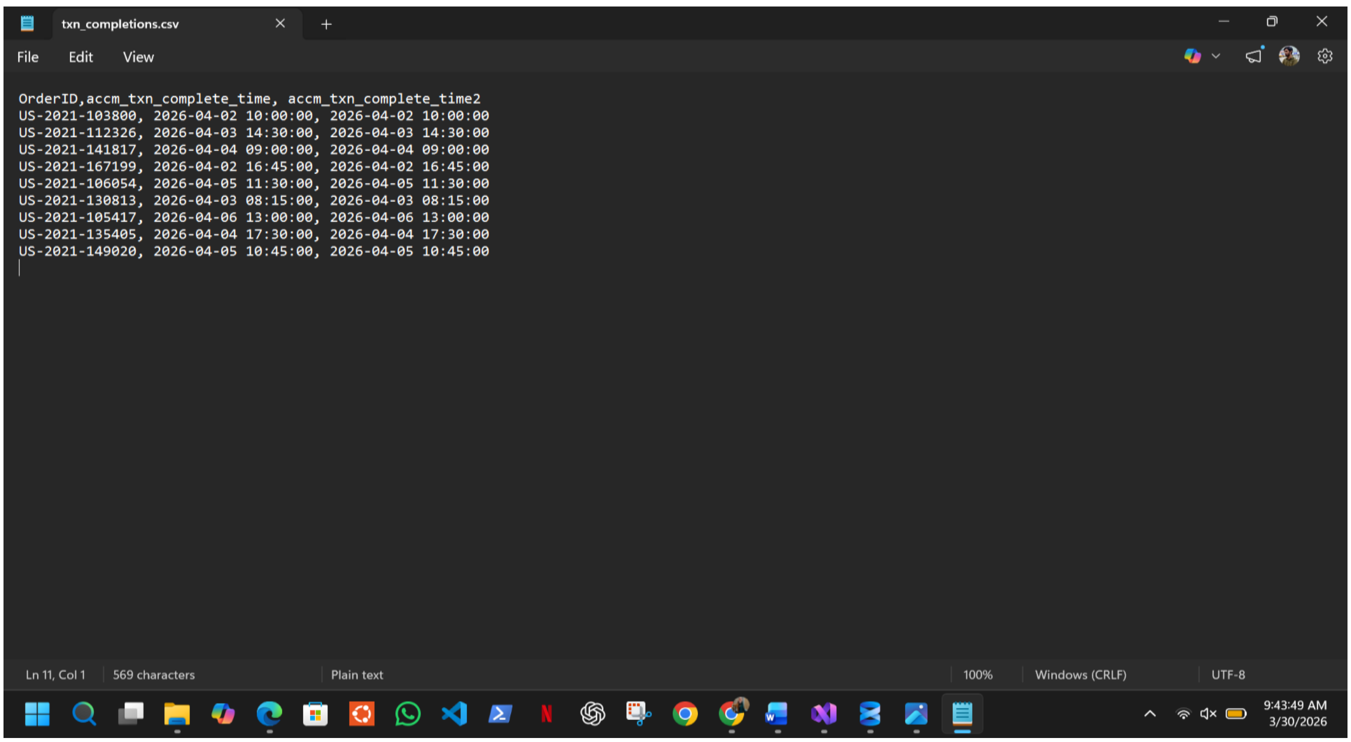
- **accm_txn_create_time** - records the timestamp when the order record was first loaded into the data warehouse. This is set to GETDATE() during the initial FactSales load.
- **accm_txn_complete_time** - records the timestamp when the order was completed. This is initially NULL and gets updated later by a separate ETL package.
- **txn_process_time_hours** - the calculated difference in hours between accm_txn_complete_time and accm_txn_create_time, representing how long the order took to be processed.

6.3 Update_AccumulatingFact.dtsx

A separate SSIS package was created to simulate the arrival of completion data and update the fact table accordingly. The package reads from txn_completions.csv - a flat file containing OrderIDs and their corresponding completion timestamps - and updates the matching rows in FactSales.

The package contains two components in the Control Flow:

- **Data Flow Task** - reads txn_completions.csv via a Flat File Source, converts data types using a Data Conversion transformation, and uses an OLE DB Command to execute a parameterized UPDATE statement that sets accm_txn_complete_time for each matching OrderID.
- **Execute SQL Task** - runs after the Data Flow and calculates txn_process_time_hours for all rows where accm_txn_complete_time is now populated, using DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time).




```
OrderID,accm_txn_complete_time, accm_txn_complete_time2
US-2021-103800, 2026-04-02 10:00:00, 2026-04-02 10:00:00
US-2021-112326, 2026-04-03 14:30:00, 2026-04-03 14:30:00
US-2021-141817, 2026-04-04 09:00:00, 2026-04-04 09:00:00
US-2021-167199, 2026-04-02 16:45:00, 2026-04-02 16:45:00
US-2021-106054, 2026-04-05 11:30:00, 2026-04-05 11:30:00
US-2021-130813, 2026-04-03 08:15:00, 2026-04-03 08:15:00
US-2021-105417, 2026-04-06 13:00:00, 2026-04-06 13:00:00
US-2021-135405, 2026-04-04 17:30:00, 2026-04-04 17:30:00
US-2021-149020, 2026-04-05 10:45:00, 2026-04-05 10:45:00
```

Figure 6.1 - txn_completions.csv containing order completion timestamps used as input for the accumulating fact update

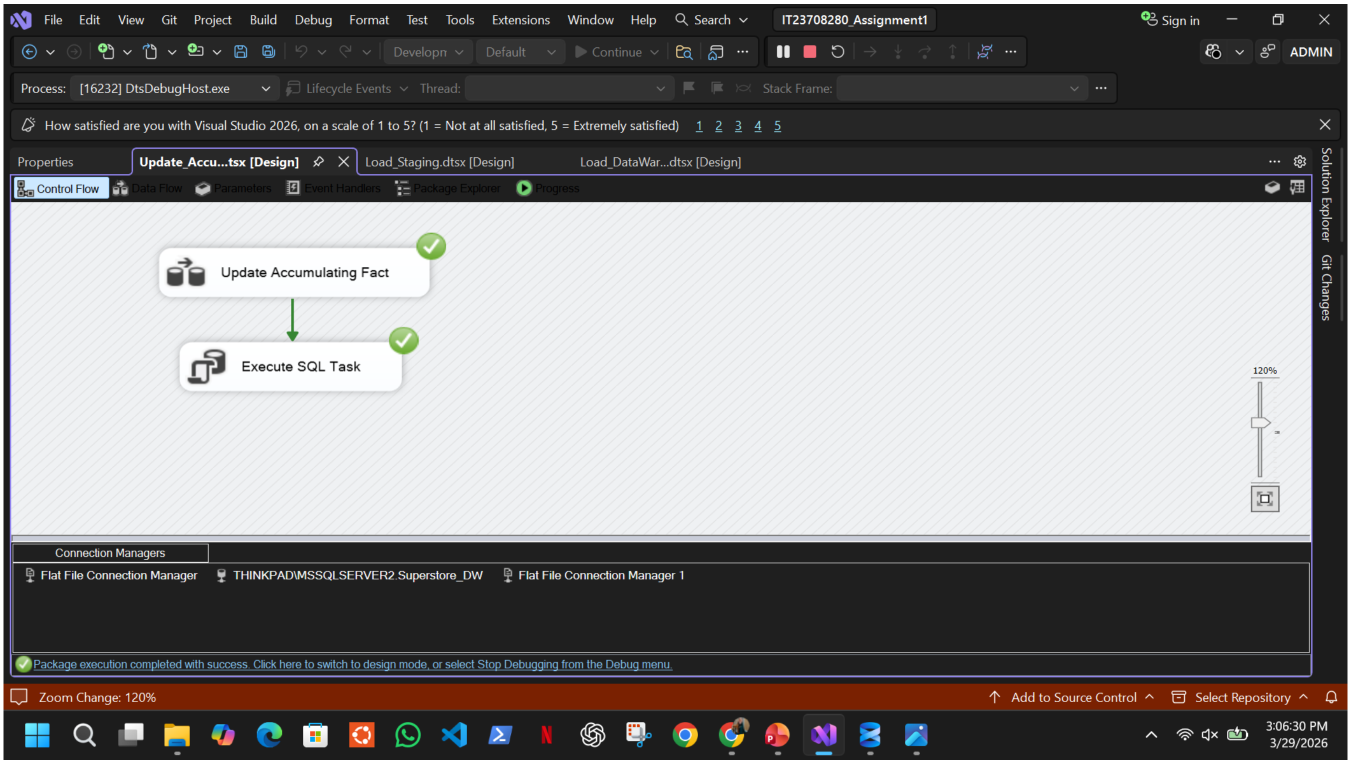


Figure 6.2 - Update_AccumulatingFact.dtsx Control Flow showing Data Flow Task updating completion time followed by Execute SQL Task calculating process hours

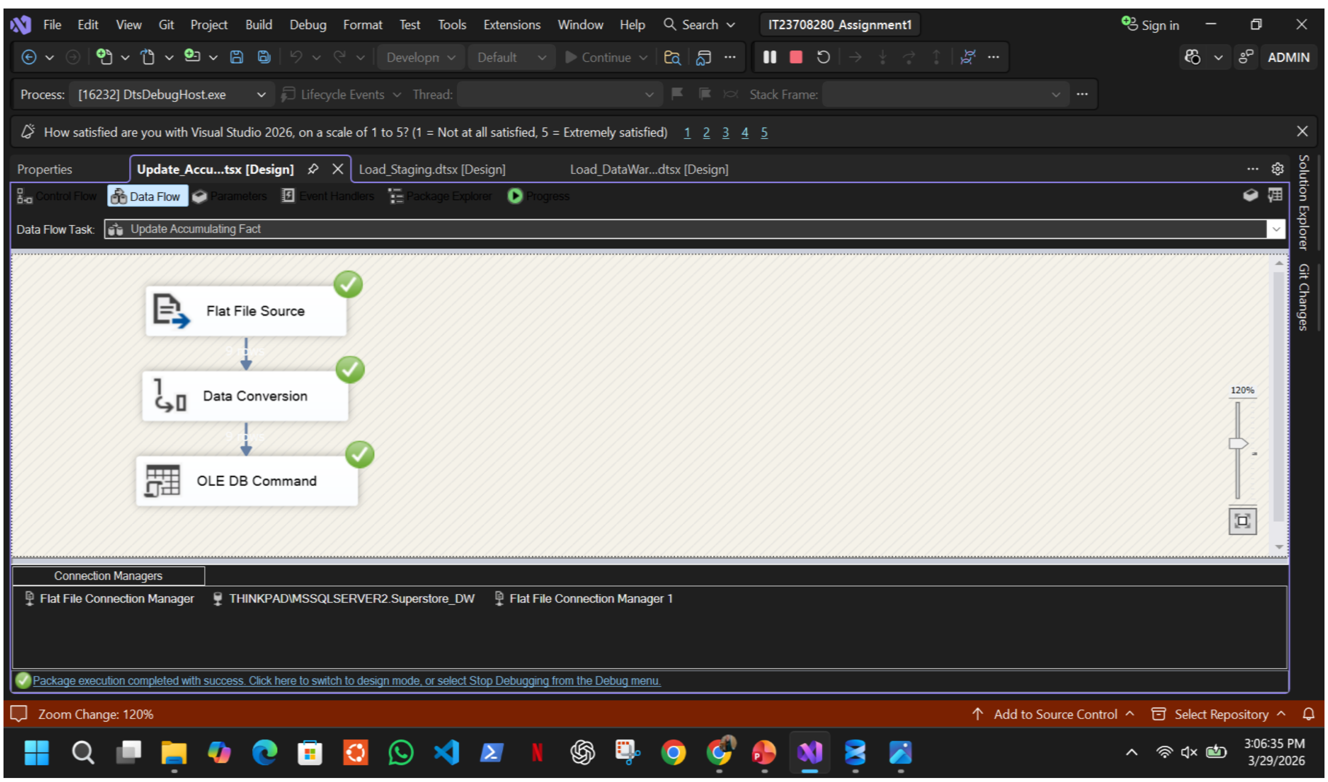


Figure 6.3 - Data Flow for accumulating fact update reading from CSV and updating FactSales rows via OLE DB Command

6.4 Validation

After running the package, a validation query confirmed that 9 unique orders were successfully updated with completion timestamps and calculated process hours (appearing as 20 rows due to multiple order line items sharing the same OrderID), while the remaining rows retain NULL values representing orders still awaiting completion - demonstrating the accumulating fact pattern correctly.

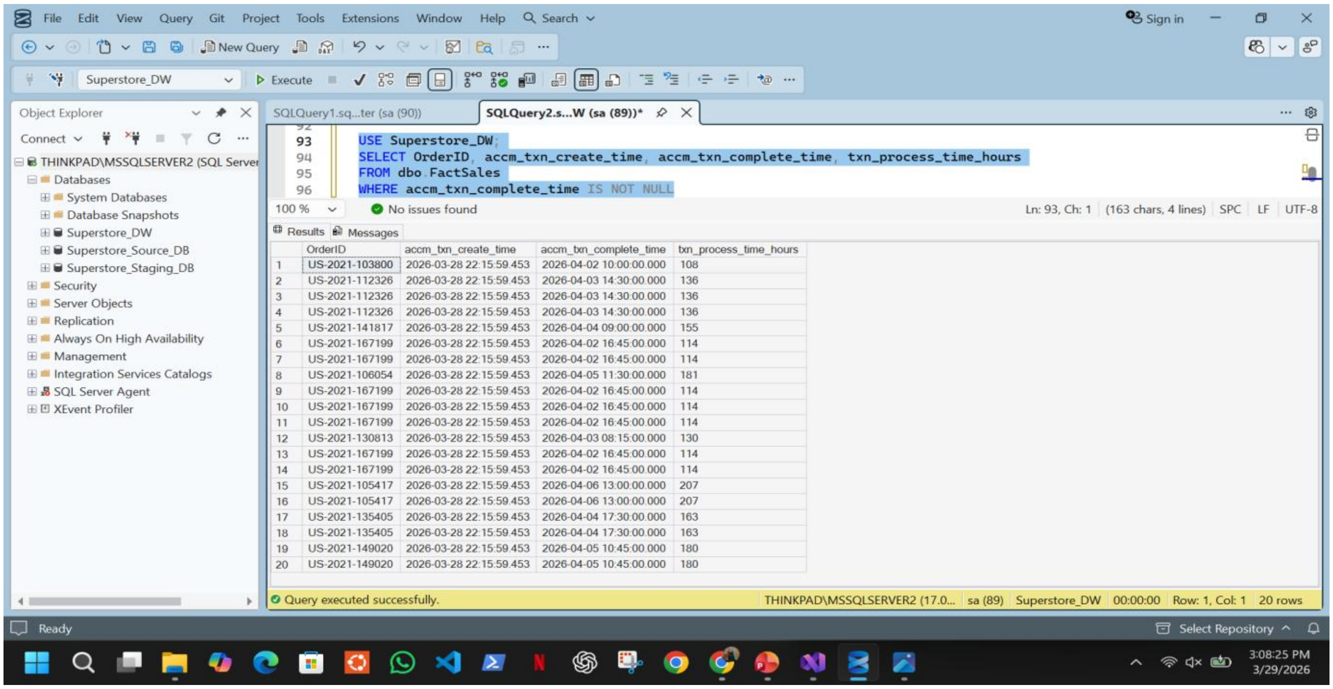


Figure 6.4 - Validation query confirming accm_txn_complete_time and txn_process_time_hours successfully updated for completed orders

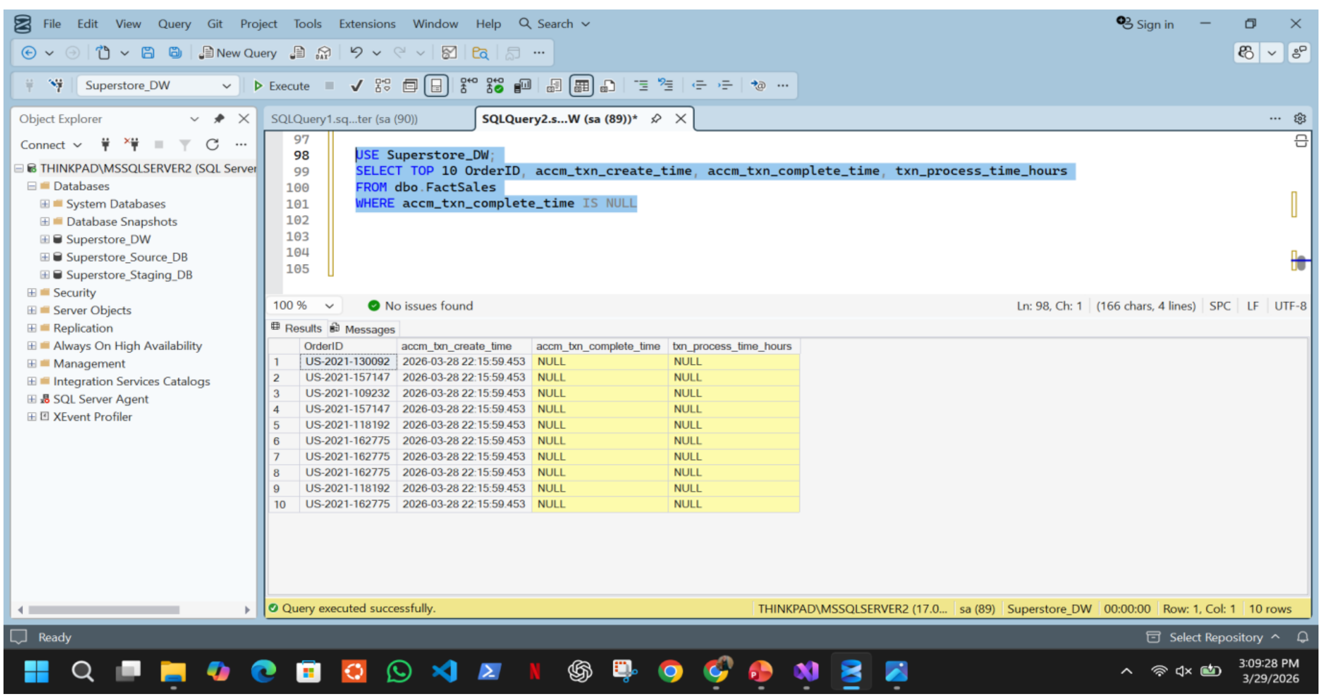


Figure 6.5 - Remaining FactSales rows with NULL completion time representing orders yet to be completed, demonstrating the accumulating fact pattern

7. References

1. Kaggle. (2021). *Superstore Sales Dataset*. Retrieved from https://www.kaggle.com/datasets/aditirai2607/super-market-dataset